Universidad Veracruzana
Desarrollo de Sistemas Web (DSW-19559)

P04. Registro individual de aportación y decisiones técnicas

Tercer incremento del proyecto PR09
Monitoreo de huerto o ambiente

Equipo DSW-E01
Alejandro Pacheco Luna (zywoxxx)

Facilitador: Dr. Gabriel Rodríguez Vásquez
Repositorio: github.com/zywoxxx/dsw-p02-monitoreo-huerto
Orizaba, Ver., 9 de octubre de 2026

1. Datos

Estudiante	Alejandro Pacheco Luna
Cuenta de GitHub	zywoxxx
Equipo	DSW-E01
Proyecto	PR09 - Monitoreo de huerto o ambiente
Actividad	P04 - Arquitectura Web 3.0 (rúbrica R04, criterio de aportación individual)
Responsabilidad en el acta	Evidencia y documentación
Repositorio	github.com/zywoxxx/dsw-p02-monitoreo-huerto

2. Qué aporté

Mi parte en el acta es evidencia y documentación, y en este incremento también hice la integración en el repositorio. Lo que sigue se puede revisar en los commits de P04, que están a nombre de mi cuenta.

Aportación	**Dónde se ve**
Contrato de la API escrito antes que el código, con las diez operaciones y el cuerpo común de error	docs/p04/openapi.yaml
Esqueleto de la API integrado, con el flujo principal funcionando	api-spring/
Esqueleto del cliente integrado, con acceso, captura e historial	frontend-angular/
Pruebas del contrato contra la API en marcha, validando cada respuesta contra su esquema	scripts/pruebas_api.py; evidencia/txt/pruebas_api.txt
Recorrido del cliente en navegador y capturas	scripts/recorrido_angular.py; evidencia/img/
Módulo M04 del verificador y prueba con la base detenida	scripts/verify-m04.sh; scripts/prueba_base_detenida.sh
Diagramas de componentes, secuencia y despliegue	scripts/diagramas_p04.py; docs/p04/diagramas/
Documentos de arquitectura, modelo, endpoints, trazabilidad, verificación e índice de evidencia	docs/p04/*.md

3. La decisión que justifico: Spring JDBC en lugar de JPA

Es la decisión D3 del documento de arquitectura del equipo. La desarrollo aquí porque es la que más afecta a lo que ya teníamos hecho en P02 y P03.

3.1 El problema

La API necesita leer sensores y lecturas y escribir una lectura con su alerta. La base ya existe: son siete tablas creadas por sql/01_schema.sql, con datos, y las usan también las aplicaciones de P02 y P03. Había que decidir cómo llega Spring Boot a esas tablas.

3.2 Las opciones

Criterio	JDBC puro (como P02 y P03)	Spring JDBC (NamedParameterJdbcTemplate)	Spring Data JPA (Hibernate)
SQL	escrito a mano	escrito a mano	lo genera Hibernate; se puede escribir con @Query
Conexiones	abrir y cerrar en cada método	las administra Spring con un pool (HikariCP)	igual
Transacciones	setAutoCommit(false), commit, rollback	@Transactional	@Transactional
Código por tabla	consulta + recorrer el ResultSet	consulta + RowMapper	clase @Entity + interfaz de repositorio
Relación con el esquema	ninguna: usa lo que haya	ninguna: usa lo que haya	mapea clases a tablas; mal configurado puede intentar crear o alterar tablas
Reutiliza el SQL de P02 y P03	sí	sí, casi sin cambios	no
Qué hay que aprender	nada nuevo	poco	entidades, relaciones, carga perezosa, estados de una entidad

3.3 Por qué Spring JDBC

- El SQL ya estaba probado. Las consultas del historial y la inserción con RETURNING id venían de P02. Con Spring JDBC pasaron a la API cambiando los ? por parámetros con nombre.
- Las respuestas de la API no son tablas. El esquema Lectura del contrato junta cinco tablas (lectura, sensor, zona, variable, alerta). Con SQL es un SELECT con joins y un RowMapper. Con JPA habría que mapear las cinco entidades con sus relaciones y luego convertirlas a otro objeto para la respuesta.
- La base es compartida. Tres aplicaciones escriben en las mismas tablas. Prefiero que ninguna herramienta tenga la posibilidad de modificar el esquema; con JDBC eso no puede pasar, y además dejamos spring.sql.init.mode=never.
- Frente al JDBC puro, quita lo repetitivo. En P03 cada repositorio abría su conexión y la transacción de lectura más alerta se manejaba a mano. Ahora la transacción es una anotación en LecturaService.registrar y el pool evita abrir una conexión por petición, que era una limitación anotada en P03.

3.4 Lo que cuesta

- Los RowMapper se escriben a mano; si se agrega una columna hay que tocar la consulta y el mapeo.

- No hay paginación automática: LecturaRepository tiene una consulta para los datos y otra para el total.
- Si el modelo creciera mucho, o hubiera muchas pantallas de alta y edición, JPA ahorraría más de lo que cuesta. Con siete tablas y dos recursos implementados no es el caso.

3.5 Cómo se comprobó que funciona

Comprobación	Evidencia
Una lectura válida suma exactamente una fila	casos 12 y 16 de pruebas_api.txt (conteo antes y después)
Una lectura fuera de umbral suma una fila en lectura y una en alerta	casos 14 y 15
Una lectura rechazada no deja nada	casos 17 a 25
Con la base detenida no se guarda nada y la API responde 503	txt/14_base_detenida.txt
El jar lleva spring-jdbc y el driver de PostgreSQL, y no lleva Hibernate	txt/03_jar_contenido.txt

4. Un error que me tocó diagnosticar

Al probar con la base detenida, GET /api/sensores respondía 503 como dice el contrato, pero POST /api/lecturas respondía 500. La diferencia es que el registro es transaccional: Spring intenta abrir la transacción antes de entrar al método, y cuando no hay conexión lanza CannotCreateTransactionException, que pertenece a la jerarquía de transacciones y no a la de acceso a datos. El manejador solo atendía DataAccessException, así que el error caía en el caso genérico. Se corrigió agregando TransactionException al mismo manejador y una prueba (LecturaControllerTest.baseNoDisponible) para que no vuelva a pasar. Lo que aprendí es que una consecuencia de usar @Transactional es que aparece un tipo de fallo que con JDBC puro no existía por separado.

```
--- POST /api/lecturas con token
{"codigo":"BASE_NO_DISPONIBLE","mensaje":"No fue posible completar la operacion en PostgreSQL; no se
guardo nada.","instante":"2026-09-29T12:42:35.114822-06:00"}
[estado 503, 5.016927 s]
```

5. Lo que no hice o queda pendiente

- No implementé zonas, alertas ni anotaciones en la API; están en el contrato y marcadas como PENDIENTE.
- No probé el despliegue en Linux ni la reproducción en otra máquina.
- Este registro es solo mío. Las aportaciones de mis compañeros las escribe cada uno en su archivo.